Отказоустойчивость Apache ZooKeeper

Старцев Иван Александрович · ИСУ 467585 · группа J3311

1. Ансамбль из трёх серверов

На MacBook установлена Java 17.0.21 и распакованы три копии ZooKeeper 3.9.6. JAVA_HOME задан при запуске. У каждой копии своя папка данных и файл myid с номером 1, 2 или 3. Список server.1, server.2, server.3 одинаковый во всех конфигурациях.

Сервер / myid	Порт клиента	Порты обмена / выборов	dataDir
zk1 / 1	2181	2881 / 3881	.runtime/zk1/data
zk2 / 2	2182	2882 / 3882	.runtime/zk2/data
zk3 / 3	2183	2883 / 3883	.runtime/zk3/data

Конфигурации находятся в conf/zk1, conf/zk2, conf/zk3. tickTime=1000, initLimit=10, syncLimit=5. Серверы запущены штатным zkServer.sh:

```
python3 zookeeper.py start
python3 zookeeper.py status
```

```
$ python3 zookeeper.py status
ZooKeeper 1: status
Mode: follower
ZooKeeper 2: status
Mode: follower
ZooKeeper 3: status
Mode: leader
exit_code=0

После остановки zk3:
{"1": "follower", "2": "leader", "3": "stopped"}

После остановки zk2 и zk3:
srvr -> 127.0.0.1:2181
This ZooKeeper instance is not currently serving requests

{"event": "three-client", "write_read": "PASS"}
{"event": "two-client", "write_read": "PASS"}
{"event": "one-client", "write_read": "UNAVAILABLE", "error": "KazooTimeoutError"}
{"event": "restored-client", "write_read": "PASS"}

После запуска zk2 и zk3:
{"1": "follower", "2": "leader", "3": "follower"}
```

При первом запуске лидером стал zk3. После остановки zk3 запись и чтение работают. После остановки ещё и zk2 оставшийся zk1 отвечает «not currently serving requests», подключение клиента заканчивается тайм-аутом. После запуска zk2 и zk3 запись и чтение снова проходят.

2. Эфемерный узел и расчёт

Открыты два клиента zkCli: первый подключён к порту 2181, второй - к 2182. В первом создан /hw4/ephemeral-demo с флагом -e. Второй видит значение alive и ненулевой ephemeralOwner. После quit в первом клиенте узел исчезает: во втором получено Node does not exist.

```
Клиент 1 (2181):
> create -e /hw4/ephemeral-demo alive
[zk: 127.0.0.1:2181(CONNECTED) 0] Created /hw4/ephemeral-demo

Клиент 2 (2182), до закрытия клиента 1:
> get -s /hw4/ephemeral-demo
[zk: 127.0.0.1:2182(CONNECTED) 0] alive
dataVersion = 0
ephemeralOwner = 0x1000815d2220002

Клиент 1:
> quit
WatchedEvent state:Closed type:None path:null zxid: -1

Клиент 2, после закрытия клиента 1:
> sync /hw4

> get -s /hw4/ephemeral-demo
[zk: 127.0.0.1:2182(CONNECTED) 1] Sync is OK
[zk: 127.0.0.1:2182(CONNECTED) 2] Node does not exist: /hw4/ephemeral-demo
```

Координаторы

Использован тот же CSV it_vacancies_2024.csv из ДЗ № 3, размер 847 619 384 байта. Его SHA-256 совпал с прошлой работой. Фильтр Python-вакансий и нормализация навыков те же, что в ДЗ № 3. Здесь считаются только пары.

Координаторы A и B подключаются ко всем трём серверам. Запрошенный и согласованный тайм-аут сессии - 10 000 мс. Лидер занимает эфемерный узел /leader и увеличивает эпоху одной транзакцией. После каждых 10 000 строк сохраняется чекпоинт: позиция в CSV, число выбранных вакансий и накопленные частоты пар.

Чекпоинты и CSV результата лежат в общей локальной папке. ZooKeeper хранит только указатели и метаданные. В обоих опытах A остановлен после строки 50 000: в чекпоинте уже 735 выбранных вакансий. B загрузил именно этот чекпоинт; пропущенные строки CSV повторно прочитал, но в подсчёт не добавлял.

Контрольное число	ДЗ № 3	Оба опыта, A и B
Строк исходного CSV	229 388	229 388
Python-вакансий	3 894	3 894
Различных пар навыков	17 801	17 801
Всего появлений пар	81 344	81 344

Сравнены все отсортированные пары и их частоты. Четыре выходных CSV побайтно совпали с baseline/hw3_pairs.csv из ДЗ № 3. SHA-256 результата:

```
f81acce1a87988f6cac84887e50b04cd27316edfefb158e155407f62b71a4956
```

3. Приостановка лидера без защиты

```
python run_experiment.py --run no-fencing --fencing 0
```

Запущены две отдельные копии координатора и получатель результата. A стал лидером с эпохой 1. Посреди расчёта ему отправлен SIGSTOP. Это останавливает весь процесс вместе с обменом сообщениями с ZooKeeper. После истечения сессии узел /leader исчез, B занял его и получил эпоху 2.

От SIGSTOP до события leader у B прошло 10.55 с. Время измерено по монотонным часам одного ноутбука. B продолжил с чекпоинта и закончил расчёт до возобновления A.

```
15:17:59.021 A leader {"epoch": 1}
15:17:59.023 A restore {"cursor": 0, "selected": 0}
15:18:03.000 SIGSTOP {"pid": 13209}
15:18:13.550 B leader {"epoch": 2}
15:18:13.559 B restore {"cursor": 50000, "selected": 735}
15:18:13.584 failover {"seconds": 10.555}
15:18:29.137 B computed {"epoch": 2, "selected": 3894, "unique_pairs": 17801, "occurrences": 81344,
"matches_hw3": true}
15:18:29.169 B записал результат {"epoch": 2, "pointer_version": 1}
15:18:43.007 SIGCONT {"pid": 13209, "pause_seconds": 40.011}
15:18:58.008 A computed {"epoch": 1, "selected": 3894, "unique_pairs": 17801, "occurrences": 81344,
"matches_hw3": true}
15:18:58.043 A записал результат {"epoch": 1, "pointer_version": 2}

ZooKeeper /hw4/no-fencing/result
{"coordinator": "A", "epoch": 1, "file": "results/no-fencing/A-pairs.csv", "sha256":
"f81acce1a87988f6cac84887e50b04cd27316edfefb158e155407f62b71a4956"}
dataVersion = 2
Количество записей /writes: 2
```

A возобновлён через 40.01 с командой SIGCONT. Он продолжил свой старый расчёт с эпохой 1 и тоже отправил результат. Без проверки эпохи получатель принял обе записи: в журналах две строки «записал результат», в /writes два узла, dataVersion указателя /result равна 2.

Итоговый указатель оказался от старого лидера A с эпохой 1, хотя текущая эпоха уже 2. Числа в этом опыте совпали, но повторная публикация всё равно произошла. Значит, одного выбора нового лидера недостаточно, чтобы остановить старую запись.

4. Защита по эпохе

```
FENCING=1 python run_experiment.py --run fencing
```

Повторён тот же опыт. A остановлен на 40.00 с; B стал лидером через 10.59 с, восстановил чекпоинт на строке 50 000 и опубликовал результат с эпохой 2. После SIGCONT публикация A с эпохой 1 отклонена с BadVersionError.

```
15:19:16.091 A leader {"epoch": 1}
15:19:16.091 A restore {"cursor": 0, "selected": 0}
15:19:20.011 SIGSTOP {"pid": 13870}
15:19:30.598 B leader {"epoch": 2}
15:19:30.598 failover {"seconds": 10.591}
15:19:30.607 B restore {"cursor": 50000, "selected": 735}
15:19:46.357 B computed {"epoch": 2, "selected": 3894, "unique_pairs": 17801, "occurrences": 81344,
"matches_hw3": true}
15:19:46.383 B записал результат {"epoch": 2, "pointer_version": 1}
15:20:00.012 SIGCONT {"pid": 13870, "pause_seconds": 40.005}
15:20:14.436 A computed {"epoch": 1, "selected": 3894, "unique_pairs": 17801, "occurrences": 81344,
"matches_hw3": true}
15:20:14.463 A BadVersionError {"epoch": 1}

ZooKeeper /hw4/fencing/result
{"coordinator": "B", "epoch": 2, "file": "results/fencing/B-pairs.csv", "sha256":
"f81acce1a87988f6cac84887e50b04cd27316edfefb158e155407f62b71a4956"}
dataVersion = 1
Количество записей /writes: 1
```

Осталась одна запись /writes и указатель от B, dataVersion=1. Все пары и частоты совпали с ДЗ № 3. FENCING управляет проверкой при публикации; устаревшие чекпоинты защищены в обоих режимах.

5. Ответы на вопросы

Почему три сервера, а не два и не четыре? Для решения нужно больше половины голосов. Из двух нужны оба: отказ одного останавливает работу. Из трёх достаточно двух, поэтому один отказ допустим. Из четырёх нужны три: переносится всё тот же один отказ, а серверов больше. Три - минимальный состав с отказоустойчивостью к одному серверу.

Почему эпоху проверяет получатель записи? ZooKeeper выбирает нового лидера, но не может запретить старому процессу отправить результат. Процесс может зависнуть, потерять сессию и потом продолжить работу. Поэтому получатель должен отклонять старую эпоху при самой записи, а не доверять словам отправителя.

В этом стенде получатель - отдельный HTTP-процесс. Он хранит указатель результата в ZooKeeper и одной транзакцией выполняет check версии /epoch, обновление /result и создание записи /writes. При старой эпохе вся транзакция отклоняется. Саму проверку версии исполняет ZooKeeper, но именно получатель обязан её запросить. Для внешнего хранилища такая защита нужна на стороне его записи.

GitHub: https://github.com/ivampo/vk_edu_lab4